# Calculator Application Documentation

## By KnitX

**Calculator Application**

---

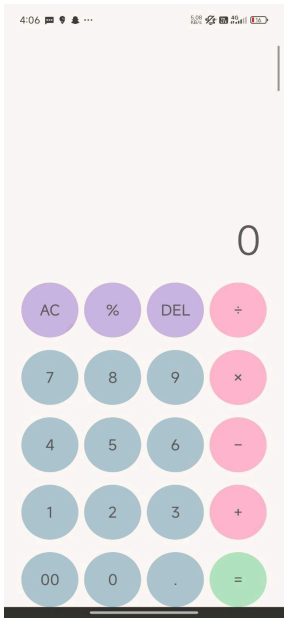
# 1. Introduction

The Calculator Application is a simple Android-based calculator developed using Kotlin in Android Studio. The application performs basic arithmetic operations such as addition, subtraction, multiplication, division, percentage calculation, decimal operations, and deletion functionalities.

The project is designed with a clean user interface and provides real-time mathematical calculations through button-based interactions.

This application demonstrates:

- Android app development using Kotlin
- Event-driven programming
- UI handling using View Binding
- Arithmetic logic implementation




---

# 2. Objectives

The main objectives of this project are:

- To develop a functional calculator application for Android devices.
- To understand Android UI design using XML.
- To implement arithmetic operations using Kotlin.
- To learn event handling and button click listeners.
- To practice Android application structure and lifecycle.

---

# 3. Technologies Used

| Technology | Purpose |
|---|---|
| Kotlin | Programming language |
| Android Studio | Development environment |
| XML | User Interface Design |
| View Binding | UI component access |

| Gradle | Build automation |
|---|---|

---

# 4. Project Structure

The project contains the following major components:

```
CalculatorApplication/
│
├── app/
│   ├── src/main/java/
│   │   └── MainActivity.kt
│   │
│   ├── src/main/res/layout/
│   │   └── activity_main.xml
│   │
│   ├── src/main/res/drawable/
│   ├── src/main/AndroidManifest.xml
│   └── build.gradle.kts
│
└── Gradle Scripts
```

---

# 5. Features of the Application

The calculator application provides the following features:

## Basic Arithmetic Operations

- Addition (+)
- Subtraction (-)
- Multiplication (×)
- Division (÷)

## Additional Functionalities

- Percentage calculation
- Decimal point handling
- Double zero input (00)
- Delete last digit
- All clear functionality
- Error handling for division by zero

---

# 6. User Interface Design

The application interface is designed using XML layouts.

## UI Components

| Component | Description |
| --- | --- |
| TextView | Displays results and expressions |
| Buttons | Number and operator inputs |
| Drawable Resources | Custom button styling |

## Layout Highlights

- Calculator display area at the top
- Numeric keypad
- Operation buttons
- Clean and responsive design
- Rounded button styles using drawable XML files

---

# 7. Working Principle

The calculator works based on button click events.

## Workflow

1. User presses numeric buttons.
2. Input is displayed on the result screen.
3. User selects an arithmetic operation.
4. First number and operator are stored.
5. User enters second number.
6. When '=' is pressed, the operation is calculated.
7. Result is displayed.

---

# 8. MainActivity.kt Explanation

The `MainActivity.kt` file contains the core logic of the application.

## Variables Used

private var firstNum = 0.0
private var operator = ""
private var isNewOp = true

### Description

| Variable | Purpose |
| --- | --- |
| firstNum | Stores the first operand |
| operator | Stores selected operation |
| isNewOp | Checks whether a new input is started |

---

# 9. Number Button Handling

The application assigns click listeners to all numeric buttons.

```
buttons.forEach { btn ->
    btn.setOnClickListener {
        if (isNewOp) {
            binding.tvResult.text = btn.text
            isNewOp = false
        } else {
            binding.tvResult.append(btn.text)
        }
    }
}
```

## Functionality

- Starts a new number after operations.
- Appends digits continuously during typing.

---

# 10. Clear Functionality

The AC button resets the calculator.

```
binding.btnAC.setOnClickListener {
    binding.tvResult.text = "0"
    binding.tvExpression.text = ""
    firstNum = 0.0
    operator = ""
    isNewOp = true
}
```

## Purpose

- Clears all data.
- Resets calculator state.

---

# 11. Delete Functionality

The delete button removes the last entered character.

```
binding.btnDel.setOnClickListener {
    val str = binding.tvResult.text.toString()
    if (str.isNotEmpty() && str != "0") {
        val newStr = str.dropLast(1)
        binding.tvResult.text = if (newStr.isEmpty()) "0" else newStr
    }
}
```

## Purpose

- Allows correction of input mistakes.

---

# 12. Arithmetic Operations

The following operations are implemented:

| Operation | Symbol |
| --- | --- |
| Addition | + |
| Subtraction | - |
| Multiplication | × |

Division ÷

## Operation Preparation

private fun prepareOperation(op: String)

This function:

- Stores the first number.
- Stores the selected operator.
- Updates the expression display.

---

# 13. Calculation Logic

The `calculate()` function performs the arithmetic operation.

```
val result = when (operator) {
    "+" -> firstNum + secondNum
    "-" -> firstNum - secondNum
    "×" -> firstNum * secondNum
    "÷" -> if (secondNum != 0.0) firstNum / secondNum else Double.NaN
    else -> secondNum
}
```

## Division by Zero Handling

If division by zero occurs:

binding.tvResult.text = "Error"

This prevents application crashes.

---

# 14. Percentage Function

The percentage button divides the entered value by 100.

val res = num / 100

## Example

| Input | Output |
|---|---|
| 50 | 0.5 |
| 25 | 0.25 |

---

# 15. Decimal Handling

The decimal button ensures only one decimal point exists.

```
if (!binding.tvResult.text.contains(".")) {
    binding.tvResult.append(".")
}
```

---

# 16. Result Formatting

The `formatResult()` function removes unnecessary decimal values.

```
private fun formatResult(num: Double): String {
    return if (num == num.toLong().toDouble()) {
        num.toLong().toString()
    } else {
        num.toString()
    }
```

}

### Example

| Actual Value | Displayed Value |
|---|---|
| 5.0 | 5 |
| 5.25 | 5.25 |

---

# 17. Advantages of the Application

- Simple and user-friendly interface
- Lightweight Android application
- Fast calculation response
- Easy to understand source code
- Good beginner-level Android project

---

# 18. Limitations

- Only basic arithmetic operations are supported.
- No scientific calculator functions.
- No calculation history.
- No memory storage functions.

---

# 19. Future Enhancements

Possible future improvements include:

- Scientific calculator operations
- Dark mode support
- Calculation history
- Voice input support

- Improved animations
- Advanced mathematical functions
- Theme customization

---

# 20. Testing

The application can be tested using:

- Android Emulator
- Physical Android devices

## Test Cases

| Test Case | Expected Result |
|---|---|
| 5 + 5 | 10 |
| 10 ÷ 2 | 5 |
| 10 ÷ 0 | Error |
| 25% | 0.25 |

---

# 21. Conclusion

The Calculator Application successfully demonstrates the implementation of a basic Android calculator using Kotlin and XML. The project provides practical experience in Android development, UI handling, event listeners, and arithmetic logic implementation.

This application serves as a strong beginner project for understanding Android app development concepts and can be expanded further into a fully featured scientific calculator.

---

# 22. References


1. Android Developers Documentation
2. Kotlin Official Documentation
3. Android Studio User Guide
4. XML Layout Design Documentation
5. Gradle Build System Documentation


---

# 23. Developer Information

**Project Name:** Calculator Application
**Platform:** Android
**Programming Language:** Kotlin
**IDE:** Android Studio
**Application Type:** Mobile Calculator Application